MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Mini Project Report
of
Database Systems Lab (CSE 2241)

Web-Based Classroom Booking System

SUBMITTED
BY

STUDENT NAME	REG. NO	ROLL NO	SECTIONS
Anudeep Reddy	240905502	50	CSE C2
Sanjeet Roy	240905512	51	CSE C2
Soham Gupta	240905518	52	CSE C2

School of Computer Engineering
Manipal Institute of Technology, Manipal.
April 2026

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Manipal
08/04/2026

CERTIFICATE

This is to certify that the project titled **Web-Based Classroom Booking System** is a record of the bonafide work done by **Anudeep Reddy (Reg. No. 240905502), Sanjeet Roy (Reg. No. 240905512), Soham Gupta (Reg. No. 240905518)** submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in COMPUTER SCIENCE AND ENGINEERING of Manipal Institute of Technology, Manipal, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

Name and Signature of Examiners:

1. **Dr. Divya Rao , Associate Professor, SCE.**

Table of Contents

ABSTRACT 5
CHAPTER 1: INTRODUCTION 6
1.1 System Architecture 6
1.2 Scope of the Project 6
1.3 Tools and Technologies 6
CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES 8
2.1 Problem Statement 8
2.2 Functional Requirements 8
2.3 Objectives 9
CHAPTER 3: METHODOLOGY 10
3.1 Requirement Gathering 10
3.2 Conceptual Design 10
3.3 Logical Design & Normalization 10
3.4 Physical Design 11
3.5 Implementation 11
3.6 Testing & Validation 11
CHAPTER 4: ER DIAGRAM & RELATIONAL SCHEMA WITH SAMPLE DATA 12
4.1 Entity-Relationship Diagram 12
4.2 Key Relationships 12
4.3 Relational Schema 13
4.4 Sample Data 13
CHAPTER 5: DDL COMMANDS & PL/SQL PROCEDURES / FUNCTIONS / TRIGGERS 15
5.1 Database Initialization 15
5.2 DDL — Lookup Tables 15
5.3 DDL — Core Entity Tables 15
5.4 DDL — Relationship & Extension Tables 16
5.5 Views 18
5.6 Triggers 18
5.7 Stored Procedures & PL/SQL Package 21
5.8 User-Defined Function 22

5.9 Representative SQL Queries 23
CHAPTER 6: RESULTS & SNAPSHOTS 25
6.1 Table Creation Output 25
6.2 View Outputs 25
6.3 Trigger Verification 26
6.4 Stored Procedure & Function Results 27
6.5 Web Application Snapshots 28
CHAPTER 7: CONCLUSION, LIMITATIONS & FUTURE WORK 31
7.1 Conclusion 31
7.2 Limitations 31
7.3 Future Work 32
CHAPTER 8: REFERENCES 33

ABSTRACT

Universities and colleges face persistent challenges in coordinating classroom use across hundreds of courses, multiple departments, and thousands of students. Manual scheduling systems lead to room conflicts, missed cancellation notices, poor room utilization, and no audit trail of changes. This project presents the design and full-stack implementation of a Web-Based Classroom Booking System — a structured, role-based application built on a rigorously normalized Oracle XE relational database.

The system allows faculty to reserve classrooms for one-time or recurring (weekly) sessions, automatically notifies enrolled students via triggers when a lecture is booked or cancelled, and prevents double-booking through database-level conflict detection. Three distinct role-based dashboards — for administrators, faculty, and students — are served as a single-page HTML/JavaScript application connected to a Node.js (Express) backend API.

The database design is decomposed through all four normal forms — 1NF, 2NF, 3NF, and BCNF — yielding a sixteen-table schema with atomic columns, no transitive dependencies, and no structural NULLs. The schema implements six triggers (conflict guard, faculty schedule conflict, booking notification, cancellation notification, periodic date validation, and status audit), one PL/SQL package (Booking_Manager) containing two procedures (Smart_Book and Cancel_Booking), one standalone procedure (Generate_Course_Roster using explicit parameterized cursors), one scalar function (GET_FACULTY_WORKLOAD), and three views (BookingDetails, RoomUsage, Dept_Capacity_Report). The project demonstrates the full spectrum of relational database design principles applied to a real-world academic scheduling domain.

CHAPTER 1: INTRODUCTION

Academic institutions require robust, role-aware systems to manage the full lifecycle of classroom delivery — from room reservation and conflict detection through lecture cancellation, student notification, and administrative oversight. Without a centralized, database-driven system, common failure modes include double-booked rooms, students unaware of cancelled classes, and no reliable history of who reserved which room and when.

The Web-Based Classroom Booking System addresses these problems by providing a structured relational database backend that enforces role-based access control and automates key academic scheduling workflows. The system applies core DBMS principles including entity-relationship modelling, relational schema design, normalization to BCNF, and advanced PL/SQL features such as stored procedures, packages, triggers, user-defined functions, and views to automate business logic at the database level.

1.1 System Architecture

The application follows a standard three-tier architecture: an HTML/CSS/JavaScript presentation tier, a Node.js (Express) application tier, and an Oracle XE data tier.

The presentation tier is a single-page HTML5 application (index.html) styled with custom CSS (style.css) and driven by vanilla JavaScript (script.js). The UI provides role-adaptive navigation and views for administrators, faculty, and students, all served as static files directly from the Express server.

The application tier is a Node.js server (server.js) using the Express framework. All database connectivity is handled via the oracledb npm package with parameterized queries throughout — no raw SQL string concatenation exists in the codebase. The server exposes REST endpoints under paths such as /rooms, /book, /cancel, /login, /timetable, /notifications, /courses, /student/*, and /admin/*.

The data tier is an Oracle XE database using the system schema. All domain constraints, conflict detection, and notification dispatch are enforced at this tier via triggers and foreign keys, independent of the application layer.

1.2 Scope of the Project

- Design and implementation of a normalized relational database schema with 16 tables (6 lookup, 4 core entity, 4 relationship, 1 audit, 1 extension).
- Six triggers enforcing business rules at the database level.
- One PL/SQL package (Booking_Manager) with two procedures, one standalone procedure (Generate_Course_Roster), and one scalar function (GET_FACULTY_WORKLOAD).
- Three views for reporting: BookingDetails, RoomUsage, and Dept_Capacity_Report.
- Representative SQL queries demonstrating JOINs, GROUP BY, HAVING, CTEs, and subqueries.
- Role-adaptive single-page web application with three role-based dashboards demonstrating live Oracle connectivity.
- Bulk data generation using PL/SQL loops for 20 faculty, 150 students, and programmatic enrollment seeding.

1.3 Tools and Technologies

Component	Technology
Database	Oracle XE (Express Edition) — system schema
Query Language	SQL / PL/SQL (Triggers, Procedures, Package, Function, Views)
Application Framework	Node.js with Express.js
Database Access	oracledb npm package (parameterized queries)
Frontend	Vanilla HTML5 / CSS3 / JavaScript (single-page application)
Server	Express static file serving + REST API on port 3000

CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

University campuses face a recurring challenge in coordinating classroom use across dozens of departments, hundreds of courses, and thousands of students. In the absence of a structured database system, the following issues commonly arise:

- Double-booking: Two faculty members independently reserve the same room for overlapping times.
- Faculty schedule conflicts: A faculty member is inadvertently assigned to two rooms simultaneously.
- Missed notifications: Students are not informed when a booked lecture is later cancelled.
- No audit trail: There is no reliable record of who updated or deleted a booking, making dispute resolution impossible.
- Underutilized rooms: Without visibility into a room's schedule, available slots go unbooked while other rooms are oversubscribed.

There is a need for a structured relational database system that can: enforce conflict-free room reservations at the data layer; maintain a normalized, consistent representation of users, rooms, courses, and bookings; automatically notify enrolled students on booking creation and cancellation; and maintain an immutable audit log of all booking changes.

2.2 Functional Requirements

FR1: User Authentication & Role Enforcement. The system must support three user roles — Admin, Faculty, and Student — enforced at the database level via the Roles lookup table and role_id FK on the Users table. Login is validated by matching email and password fields.

FR2: Classroom Management. Administrators must be able to register classrooms with building, room number, capacity, projector availability, and air-conditioning status. The system must prevent capacity from being set to zero or negative (CHECK constraint).

FR3: Course & Enrollment Management. Courses must be linked to departments via a foreign key. Students must be able to enroll in courses per semester. Duplicate enrollments must be prevented by a UNIQUE(student_id, course_id, semester_id) constraint.

FR4: Booking Management. Faculty must be able to create one-time or recurring (weekly) bookings via the /book endpoint. For recurring bookings, the day-of-week and recurrence end date are stored in the linked PeriodicDetails table, preventing structural NULLs in the main Bookings table.

FR5: Conflict Prevention. The database must prevent both room double-booking (prevent_overlap trigger) and faculty schedule conflicts (faculty_schedule_conflict trigger) automatically on every INSERT into Bookings.

FR6: Notification Dispatch. An AFTER INSERT trigger on Bookings (notify_on_booking) must automatically notify the booking faculty and all enrolled students. An AFTER UPDATE trigger on Bookings (notify_cancel) must automatically notify faculty and students when a booking is cancelled.

FR7: Audit Logging. Every UPDATE or DELETE on Bookings must be automatically logged to the Booking_Audit table via an AFTER UPDATE OR DELETE trigger, providing an immutable history.

FR8: Reporting & Admin. Three pre-built views must provide: all active confirmed bookings with faculty and room details (BookingDetails); per-room booking count (RoomUsage); and a department-

level capacity report with ROLLUP grouping (Dept_Capacity_Report). Admin endpoints expose total booking, cancellation, and room counts, plus per-course faculty and room statistics.

2.3 Objectives

- Design a normalized relational database schema to BCNF for managing classroom booking operations.
- Implement all 16 tables with appropriate PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK constraints.
- Develop basic SQL queries (SELECT, INSERT, UPDATE, DELETE) for all entities.
- Write complex queries using JOIN, GROUP BY, HAVING, aggregate functions, and subqueries.
- Implement 1 PL/SQL package (Booking_Manager) with 2 procedures encapsulating booking and cancellation logic.
- Implement 1 standalone stored procedure (Generate_Course_Roster) demonstrating explicit parameterized cursors.
- Implement 1 scalar function (GET_FACULTY_WORKLOAD) for computing total teaching hours.
- Implement 6 triggers enforcing business rules automatically at the database level.
- Define 3 views for dashboard display and reporting.
- Develop a role-adaptive single-page web application demonstrating live Oracle database connectivity.

CHAPTER 3: METHODOLOGY

The project follows the standard Database Development Life Cycle (DDLC) consisting of five phases: requirement gathering, conceptual design, logical design and normalization, physical implementation, and testing and validation.

3.1 Requirement Gathering

System requirements were identified by analysing the functional needs of a university classroom scheduling platform. Entities, attributes, relationships, and business rules were documented based on common academic workflows: user role management, room reservation by faculty, recurring lecture scheduling, individual lecture cancellation, student notification, and administrative oversight. The key business rules that informed the database design were:

- A room may not be double-booked for the same date and overlapping time window.
- A faculty member may not be scheduled in two rooms simultaneously on the same date.
- Only faculty or admin users may create bookings (enforced at application layer via login session).
- Recurring bookings recur on a specific day of the week between a start date and an end date.
- When a booking is confirmed, all students enrolled in that course must be notified automatically.
- When a booking is cancelled, all students enrolled in that course must be notified automatically.
- Every booking UPDATE or DELETE must be recorded in an audit table with a timestamp.

3.2 Conceptual Design

An Entity-Relationship (ER) diagram was created to represent the primary entities and their relationships. The initial conceptual model identified seven core entities: Users, Classrooms, Courses, Bookings, Cancellations, Enrollments, and Notifications.

Relationships between entities were defined along with their cardinalities:

- Users (1) → Bookings (N): One faculty member creates many bookings.
- Classrooms (1) → Bookings (N): One classroom is the subject of many bookings.
- Courses (1) → Bookings (N): One course can have many lecture bookings.
- Bookings (1) → Cancellations (N): One booking can have many individual cancellation records.
- Users (1) → Enrollments (N): One student can be enrolled in many courses.
- Courses (1) → Enrollments (N): One course has many enrolled students.
- Bookings (1) → PeriodicDetails (0..1): One recurring booking has exactly one recurrence detail record.
- Users (1) → Notifications (N): One user receives many notifications.
- Bookings (1) → Booking_Audit (N): One booking can generate many audit log entries.

3.3 Logical Design & Normalization

The ER model was converted into a relational schema. Normalization was applied systematically through all four normal forms:

1NF: All column values are atomic. The original Bookings table stored recurrence_pattern as a free-text field combining day-of-week and end-date information — a multi-value column. This was split into a separate PeriodicDetails table (extension), with day-of-week normalized to a DaysOfWeek FK. The original Classrooms table stored amenities as a comma-separated string; this was replaced with atomic boolean columns (has_projector, has_ac).

2NF: All original tables used single-column surrogate primary keys, so partial functional dependency on a composite PK cannot arise. 2NF is trivially satisfied throughout.

3NF: The original Courses table stored dept_name as a VARCHAR. The functional dependency dept_name → building holds — building is determined by dept_name, not by course_id. This is a transitive dependency (course_id → dept_name → building) violating 3NF. The fix was to extract a Departments table and replace dept_name in Courses with a dept_id foreign key.

BCNF: Free-string domain columns were identified where a non-key attribute determined its own meaning without being a superkey of its table: Users.role (plain text), Bookings.booking_type, Bookings.status, and PeriodicDetails.day_of_week. Each was extracted into its own lookup table (Roles, BookingTypes, BookingStatuses, NotifTypes, DaysOfWeek, Semesters), making the string value the primary key of its own table and replacing the column with a foreign key reference.

3.4 Physical Design

Tables were created in Oracle XE using SQL DDL. Identity columns (GENERATED BY DEFAULT AS IDENTITY) were used for auto-incrementing surrogate primary keys. ON DELETE CASCADE was applied to dependent child records (PeriodicDetails, Cancellations, Enrollments). CHECK constraints enforce capacity > 0 for Classrooms and credits > 0 for Courses. A UNIQUE constraint on (student_id, course_id, semester_id) prevents duplicate enrollments.

3.5 Implementation

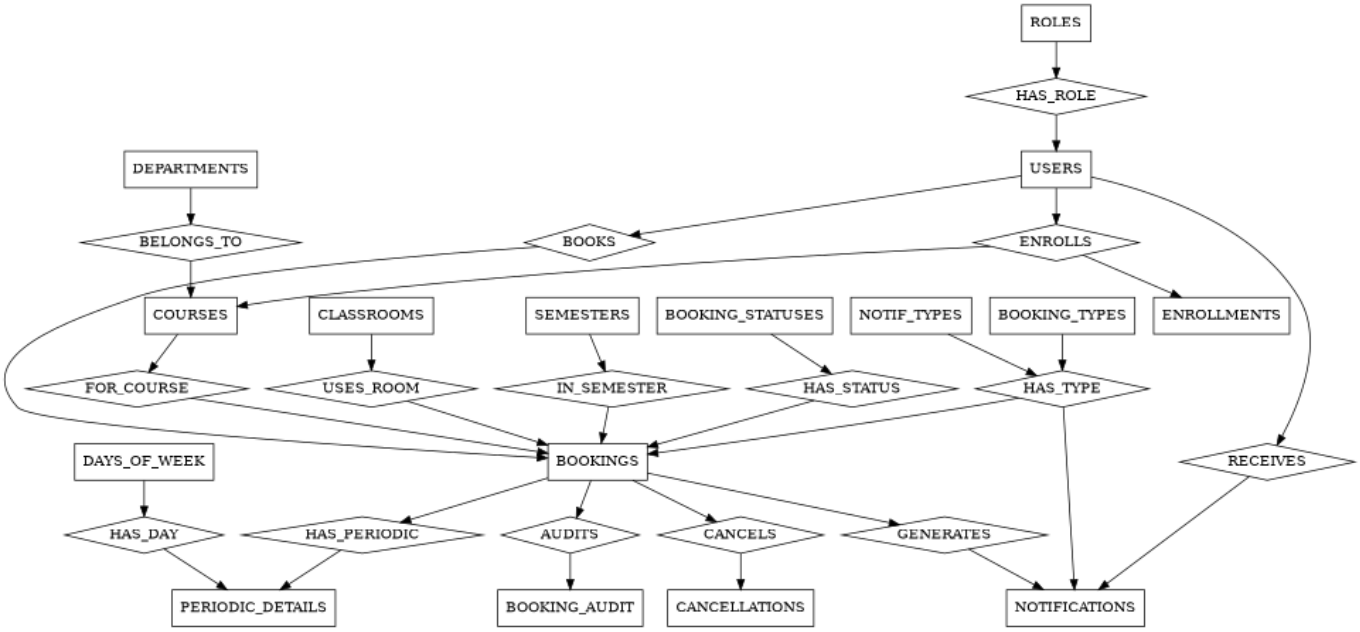
Sample data was inserted covering 3 departments, 3 booking statuses, 2 booking types, 7 days of week, 2 notification types, 1 semester, 4 manually inserted users (1 admin, 2 faculty, 1 student), 20 generated faculty members, 150 generated students, 4 courses, 5 classrooms, and programmatic enrollment seeding using PL/SQL loops. Six triggers, one function, one package (two procedures), one standalone procedure, and three views were implemented. The Node.js/Express server provides REST endpoints consumed by the single-page frontend.

3.6 Testing & Validation

All constraints were tested by attempting invalid insertions. Trigger behaviour was verified — for example, inserting a Booking for a room already confirmed at an overlapping time raises ORA-20001 from the prevent_overlap trigger; inserting a Booking where the faculty is already teaching at that time raises ORA-20004 from faculty_schedule_conflict. The notify_on_booking trigger was verified to insert Notification rows for both the faculty and all enrolled students on a new booking INSERT. The notify_cancel trigger was verified to fire on UPDATE when status_id changes to 3. The audit_bookings trigger was verified to create a Booking_Audit row on every UPDATE or DELETE. The check_periodic_dates trigger was verified to reject PeriodicDetails rows with an end_date earlier than SYSDATE.

CHAPTER 4: ER DIAGRAM & RELATIONAL SCHEMA WITH SAMPLE DATA

4.1 Entity-Relationship Diagram



4.2 Key Relationships

Relationship	Cardinality	Description
Roles → Users	1 : N	One role (Admin/Faculty/Student) is assigned to many users via role_id FK
Departments → Courses	1 : N	One department offers many courses via dept_id FK
Users → Bookings	1 : N	One faculty member creates many bookings (faculty_id FK)
Classrooms → Bookings	1 : N	One classroom is reserved in many bookings (room_id FK)
Courses → Bookings	0..1 : N	A booking may or may not be linked to a course (course_id nullable FK)
BookingTypes → Bookings	1 : N	One booking type (One-time/Recurring) classifies many bookings
BookingStatuses → Bookings	1 : N	One status (Confirmed/Pending/Cancelled) labels many bookings
Semesters → Bookings	0..1 : N	A booking may optionally belong to a semester
Bookings → PeriodicDetails	1 : 0..1	One recurring booking has at most one recurrence detail record
DaysOfWeek → PeriodicDetails	1 : N	One day-of-week is referenced by many periodic bookings
Bookings → Cancellations	1 : N	One booking can have many cancellation records
Users → Enrollments	1 : N	One student has many course enrollments
Courses → Enrollments	1 : N	One course has many enrolled students
Semesters → Enrollments	1 : N	One semester contains many course enrollments
Users → Notifications	1 : N	One user receives many notifications
NotifTypes → Notifications	1 : N	One notification type classifies many notifications
Bookings → Booking_Audit	1 : N	One booking generates many audit log entries on UPDATE/DELETE

4.3 Relational Schema

The following relational tables were derived from the ER diagram. (* = UNIQUE constraint, # = Foreign Key, PK = Primary Key):

Roles (role_id PK, role_name*)
BookingTypes (type_id PK, type_name)
BookingStatuses (status_id PK, status_name)
NotifTypes (notif_type_id PK, type_name)
DaysOfWeek (day_id PK, day_name)
Semesters (semester_id PK, name, start_date, end_date)

Departments (dept_id PK, name)
Users (user_id PK, name, email*, role_id#, phone, password, role, created_at)
Classrooms (room_id PK, building, room_number, capacity, has_projector, has_ac)
Courses (course_id PK, title, dept_id#, credits)

Bookings (booking_id PK, faculty_id#, room_id#, course_id#, booking_type_id#, booking_date, start_time, end_time, semester_id#, status_id#, created_at)
PeriodicDetails (periodic_id PK, booking_id#, day_id#, end_date)
Enrollments (enrollment_id PK, student_id#, course_id#, semester_id#) -- UNIQUE(student_id,course_id,semester_id)
Cancellations (cancel_id PK, booking_id#, cancelled_by#, cancelled_date, reason)
Notifications (notif_id PK, user_id#, message, notif_type_id#, is_read, created_at)
Booking_Audit (audit_id PK, booking_id#, action, changed_at)

4.4 Sample Data

Users Table (Named Test Users)

user_id	name	email	role_id	role
1	Admin User	admin@manipal.edu	1	ADMIN
2	Dr. Arnab	arnab@manipal.edu	2	FACULTY
3	Prof. Sarah	sarah@manipal.edu	2	FACULTY
4	Student Test	student@manipal.edu	3	STUDENT
5-24	Faculty 1-20	faculty{i}@manipal.edu	2	FACULTY
25-174	Student 1-150	student{i}@manipal.edu	3	STUDENT

Classrooms Table

room_id	building	room_number	capacity	has_projector	has_ac
1	AB1	101	60	1	1
2	AB1	102	40	1	1
3	AB2	301	120	1	1
4	AB5	LH1	200	1	1
5	IC	Lab A	30	0	1

Courses Table

course_id	title	dept_id	credits
ICT3101	Computer Networks	1 (CS)	4
ICT3102	Embedded Systems	1 (CS)	4
MAT2101	Engineering Maths	2 (Maths)	3
DBS202	Database Systems	1 (CS)	3

Enrollment Seeding Logic

All 150 programmatically generated students (student1 through student150) are automatically enrolled in DBS202 via a PL/SQL loop. Even-ID students additionally receive ICT3102. Every third student additionally receives MAT2101.

CHAPTER 5: DDL COMMANDS & PL/SQL PROCEDURES / FUNCTIONS / TRIGGERS

5.1 Database Initialization

The project targets Oracle XE (Express Edition) using the system user. All existing tables are dropped with CASCADE CONSTRAINTS PURGE before recreating the schema to allow clean re-initialization.

```
DROP TABLE Booking_Audit CASCADE CONSTRAINTS PURGE;
DROP TABLE Notifications CASCADE CONSTRAINTS PURGE;
-- ... (all 16 tables dropped in reverse dependency order)
```

5.2 DDL — Lookup Tables

Table 1: Roles

```
CREATE TABLE Roles (
   role_id   NUMBER PRIMARY KEY,
   role_name VARCHAR2(50) NOT NULL
);
```

Tables 2–6: BookingStatuses, BookingTypes, DaysOfWeek, NotifTypes, Semesters

These follow the same pattern — a numeric primary key and a name/label column. BookingStatuses seeds three values: Confirmed (1), Pending (2), Cancelled (3). BookingTypes seeds One-time (1) and Recurring (2). DaysOfWeek seeds Sunday through Saturday (1-7). NotifTypes seeds Booking Confirmation (1) and Cancellation Alert (2). Semesters seeds one record: Semester 1 2025.

5.3 DDL — Core Entity Tables

Table 7: Departments

```
CREATE TABLE Departments (
   dept_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   name    VARCHAR2(100)
);
```

Departments was extracted from the original Courses table to eliminate the transitive dependency course_id → dept_name → building (3NF violation).

Table 8: Users

```
CREATE TABLE Users (
   user_id    NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   name       VARCHAR2(100) NOT NULL,
   email      VARCHAR2(100) UNIQUE NOT NULL,
   role_id    NUMBER REFERENCES Roles(role_id),
   phone      VARCHAR2(15),
   password   VARCHAR2(100) NOT NULL,
   role       VARCHAR2(20),
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Note: the role column stores the string label (ADMIN/FACULTY/STUDENT) alongside the normalized role_id FK. The application login query reads the role column directly for session role detection.

Table 9: Classrooms

```
CREATE TABLE Classrooms (
   room_id       NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   building      VARCHAR2(50),
   room_number   VARCHAR2(10) NOT NULL,
   capacity      NUMBER CHECK (capacity > 0),
   has_projector NUMBER(1) DEFAULT 0,
   has_ac        NUMBER(1) DEFAULT 0
);
```

The original amenities TEXT column was replaced with atomic boolean flags (has_projector, has_ac), satisfying 1NF. A CHECK constraint enforces non-zero capacity.

Table 10: Courses

```
CREATE TABLE Courses (
   course_id VARCHAR2(20) PRIMARY KEY,
   title     VARCHAR2(100) NOT NULL,
   dept_id   NUMBER REFERENCES Departments(dept_id),
   credits   NUMBER CHECK (credits > 0)
);
```

5.4 DDL — Relationship & Extension Tables

Table 11: Bookings

```
CREATE TABLE Bookings (
   booking_id      NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
   faculty_id      NUMBER NOT NULL REFERENCES Users(user_id),
   room_id         NUMBER NOT NULL REFERENCES Classrooms(room_id),
   course_id       VARCHAR2(20) REFERENCES Courses(course_id),
   booking_type_id NUMBER REFERENCES BookingTypes(type_id),
   booking_date    DATE NOT NULL,
   start_time      DATE NOT NULL,
   end_time        DATE NOT NULL,
   semester_id     NUMBER REFERENCES Semesters(semester_id),
   status_id       NUMBER DEFAULT 1 REFERENCES BookingStatuses(status_id),
   created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   CONSTRAINT chk_times CHECK (end_time > start_time)
);
```

In Oracle XE, start_time and end_time are stored as DATE columns (which include time component). The CHECK constraint enforces valid time ordering. course_id is nullable to allow non-course one-time room reservations.

Table 12: PeriodicDetails — 1:N extension of Bookings

```
CREATE TABLE PeriodicDetails (
   periodic_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
```

```
    booking_id  NUMBER NOT NULL REFERENCES Bookings(booking_id) ON DELETE
CASCADE,
    day_id      NUMBER NOT NULL REFERENCES DaysOfWeek(day_id),
    end_date    DATE NOT NULL
);
```

Only recurring bookings get a row here, eliminating structural NULLs from the main Bookings table. The end_date must be today or in the future (enforced by the check_periodic_dates trigger). The day-of-week is auto-derived from the booking date's day in the Express server and stored as a day_id FK.

Table 13: Enrollments

```
CREATE TABLE Enrollments (
    enrollment_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    student_id    NUMBER NOT NULL REFERENCES Users(user_id),
    course_id     VARCHAR2(20) NOT NULL REFERENCES Courses(course_id),
    semester_id   NUMBER REFERENCES Semesters(semester_id),
    CONSTRAINT uq_enrollment UNIQUE (student_id, course_id, semester_id)
);
```

The UNIQUE(student_id, course_id, semester_id) constraint prevents a student from enrolling in the same course twice in the same semester. The application layer also performs a pre-check before inserting to provide a user-friendly error message.

Table 14: Cancellations

```
CREATE TABLE Cancellations (
    cancel_id      NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    booking_id     NUMBER NOT NULL REFERENCES Bookings(booking_id),
    cancelled_by   NUMBER,
    cancelled_date DATE DEFAULT SYSDATE,
    reason         VARCHAR2(200)
);
```

Table 15: Notifications

```
CREATE TABLE Notifications (
    notif_id      NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    user_id       NUMBER NOT NULL,
    message       VARCHAR2(200) NOT NULL,
    notif_type_id NUMBER REFERENCES NotifTypes(notif_type_id),
    is_read       NUMBER(1) DEFAULT 0,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Table 16: Booking_Audit

```
CREATE TABLE Booking_Audit (
    audit_id   NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    booking_id NUMBER,
    action     VARCHAR2(20),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Booking_Audit is populated exclusively by the audit_bookings AFTER UPDATE OR DELETE trigger. The action column stores either 'UPDATE' or 'DELETE', providing an immutable audit trail of every modification to the Bookings table.

5.5 Views

View 1: BookingDetails — Active bookings with faculty name and room details

```
CREATE OR REPLACE VIEW BookingDetails AS
SELECT
   b.booking_id,
   u.name,
   c.room_number,
   b.booking_date,
   b.start_time,
   b.end_time,
   s.status_name
FROM Bookings b
JOIN Users u ON b.faculty_id = u.user_id
JOIN Classrooms c ON b.room_id = c.room_id
JOIN BookingStatuses s ON b.status_id = s.status_id
WHERE b.status_id != 3;
```

View 2: RoomUsage — Per-room booking count

```
CREATE OR REPLACE VIEW RoomUsage AS
SELECT
   room_id,
   COUNT(*) AS total_bookings
FROM Bookings
GROUP BY room_id;
```

View 3: Dept_Capacity_Report — Department-level capacity with ROLLUP

```
CREATE OR REPLACE VIEW Dept_Capacity_Report AS
SELECT
   NVL(d.name, 'All Departments') AS department,
   NVL(c.building, 'All Buildings') AS building,
   SUM(c.capacity)              AS total_seat_capacity,
   COUNT(c.room_id)             AS total_rooms
FROM Classrooms c
LEFT JOIN Bookings b   ON c.room_id  = b.room_id
LEFT JOIN Courses cr   ON b.course_id = cr.course_id
LEFT JOIN Departments d ON cr.dept_id = d.dept_id
GROUP BY ROLLUP(d.name, c.building);
```

The ROLLUP operator produces subtotals per department and a grand total row, providing a hierarchical capacity summary that is not achievable with a plain GROUP BY.

5.6 Triggers

Trigger 1: prevent_overlap (BEFORE INSERT on Bookings)

```
CREATE OR REPLACE TRIGGER prevent_overlap
BEFORE INSERT ON Bookings FOR EACH ROW
```

```
DECLARE
   v_count NUMBER;
BEGIN
   SELECT COUNT(*) INTO v_count
   FROM Bookings
   WHERE room_id      = :NEW.room_id
     AND booking_date = :NEW.booking_date
     AND status_id   != 3
     AND start_time   < :NEW.end_time
     AND end_time     > :NEW.start_time;
   IF v_count > 0 THEN
      RAISE_APPLICATION_ERROR(-20001, 'Room already booked for that time slot.');
   END IF;
END;
```

This trigger fires before every INSERT on Bookings and checks for time overlap against all non-cancelled bookings in the same room on the same date. It raises ORA-20001, aborting the INSERT if a conflict exists.

Trigger 2: faculty_schedule_conflict (BEFORE INSERT on Bookings)

```
CREATE OR REPLACE TRIGGER faculty_schedule_conflict
BEFORE INSERT ON Bookings FOR EACH ROW
DECLARE
   v_faculty_conflict NUMBER;
BEGIN
   SELECT COUNT(*) INTO v_faculty_conflict
   FROM Bookings
   WHERE faculty_id   = :NEW.faculty_id
     AND booking_date = :NEW.booking_date
     AND status_id   != 3
     AND start_time   < :NEW.end_time
     AND end_time     > :NEW.start_time;
   IF v_faculty_conflict > 0 THEN
      RAISE_APPLICATION_ERROR(-20004,
         'Faculty schedule conflict: already teaching in another room at this time.');
   END IF;
END;
```

This trigger prevents a faculty member from being double-booked across two different rooms at the same time, which the room-level trigger alone cannot detect.

Trigger 3: notify_on_booking (AFTER INSERT on Bookings)

```
CREATE OR REPLACE TRIGGER notify_on_booking
AFTER INSERT ON Bookings FOR EACH ROW
BEGIN
   -- Notify the Faculty who created the booking
   INSERT INTO Notifications (user_id, message, is_read)
   VALUES (:NEW.faculty_id,
         'Booking Confirmed for course ' || NVL(:NEW.course_id,'General')
         || ' in room ' || :NEW.room_id, 0);
```

```
    -- Broadcast to all enrolled students if course is set
    IF :NEW.course_id IS NOT NULL THEN
      FOR student_rec IN (
        SELECT student_id FROM Enrollments WHERE course_id = :NEW.course_id
      ) LOOP
        INSERT INTO Notifications (user_id, message, is_read)
        VALUES (student_rec.student_id,
            'New class scheduled: ' || :NEW.course_id
            || ' in room ' || :NEW.room_id
            || ' (' || TO_CHAR(:NEW.booking_date,'Mon DD') || ')', 0);
      END LOOP;
    END IF;
  END;
```

Trigger 4: notify_cancel (AFTER UPDATE on Bookings — WHEN status becomes Cancelled)

```
  CREATE OR REPLACE TRIGGER notify_cancel
  AFTER UPDATE ON Bookings FOR EACH ROW
  WHEN (NEW.status_id = 3 AND OLD.status_id != 3)
  BEGIN
    INSERT INTO Notifications (user_id, message, is_read)
    VALUES (:NEW.faculty_id,
        'Your booking for room ' || :NEW.room_id || ' has been cancelled.', 0);
    IF :NEW.course_id IS NOT NULL THEN
      FOR student_rec IN (
        SELECT student_id FROM Enrollments WHERE course_id = :NEW.course_id
      ) LOOP
        INSERT INTO Notifications (user_id, message, is_read)
        VALUES (student_rec.student_id,
            'ALERT: Class Cancelled for ' || :NEW.course_id
            || ' on ' || TO_CHAR(:NEW.booking_date,'Mon DD'), 0);
      END LOOP;
    END IF;
  END;
```

This trigger fires only when status_id transitions to 3 (Cancelled), preventing spurious notifications on other field updates.

Trigger 5: check_periodic_dates (BEFORE INSERT on PeriodicDetails)

```
  CREATE OR REPLACE TRIGGER check_periodic_dates
  BEFORE INSERT ON PeriodicDetails FOR EACH ROW
  BEGIN
    IF :NEW.end_date < TRUNC(SYSDATE) THEN
      RAISE_APPLICATION_ERROR(-20002,
        'End date for recurring booking must be today or in the future.');
    END IF;
  END;
```

Trigger 6: audit_bookings (AFTER UPDATE OR DELETE on Bookings)

```
  CREATE OR REPLACE TRIGGER audit_bookings
```

```
AFTER UPDATE OR DELETE ON Bookings FOR EACH ROW
DECLARE
   v_action VARCHAR2(20);
BEGIN
   IF DELETING THEN
      v_action := 'DELETE';
   ELSIF UPDATING THEN
      v_action := 'UPDATE';
   END IF;
   INSERT INTO Booking_Audit (booking_id, action)
   VALUES (NVL(:OLD.booking_id, :NEW.booking_id), v_action);
END;
```

Every time a booking is updated or deleted, this trigger writes an audit record. The NVL handles both UPDATE (where :NEW exists) and DELETE (where :NEW.booking_id is null) cases.

5.7 Stored Procedures & PL/SQL Package

Standalone Procedure: Generate_Course_Roster (demonstrates explicit parameterized cursors)

```
CREATE OR REPLACE PROCEDURE Generate_Course_Roster(p_course_id IN VARCHAR2)
IS
   CURSOR c_students(course_id_in VARCHAR2) IS
      SELECT u.name, u.email
      FROM Enrollments e
      JOIN Users u ON e.student_id = u.user_id
      WHERE e.course_id = course_id_in;
   v_student_rec c_students%ROWTYPE;
   v_count NUMBER := 0;
BEGIN
   DBMS_OUTPUT.PUT_LINE('--- Course Roster for ' || p_course_id || ' ---');
   OPEN c_students(p_course_id);
   LOOP
      FETCH c_students INTO v_student_rec;
      EXIT WHEN c_students%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE('Student: ' || v_student_rec.name
                           || ' | Email: ' || v_student_rec.email);
      v_count := v_count + 1;
   END LOOP;
   CLOSE c_students;
   DBMS_OUTPUT.PUT_LINE('Total Enrolled: ' || v_count);
END;
```

This procedure demonstrates the use of an explicit parameterized cursor — the cursor declaration accepts a parameter, is opened with an argument, fetched in a loop, and explicitly closed. It is executed at the end of the seed script as: EXEC Generate_Course_Roster('DBS202');

PL/SQL Package: Booking_Manager

The Booking_Manager package encapsulates the two primary booking-related procedures — Smart_Book and Cancel_Booking — in a single, reusable unit. The package specification declares the public interface:

```
CREATE OR REPLACE PACKAGE Booking_Manager AS
   PROCEDURE Smart_Book(
      p_user NUMBER, p_room NUMBER, p_date DATE,
      p_start DATE, p_end DATE);
   PROCEDURE Cancel_Booking(
      p_booking_id NUMBER, p_user_id NUMBER, p_reason VARCHAR2);
END Booking_Manager;
```

Smart_Book — conflict-checked booking creation

```
PROCEDURE Smart_Book(p_user NUMBER, p_room NUMBER,
                     p_date DATE, p_start DATE, p_end DATE) IS
   v_conflict NUMBER;
BEGIN
   SELECT COUNT(*) INTO v_conflict
   FROM Bookings
   WHERE room_id = p_room AND booking_date = p_date
     AND status_id != 3
     AND start_time < p_end AND end_time > p_start;
   IF v_conflict > 0 THEN
      RAISE_APPLICATION_ERROR(-20001,'Conflict detected: room already booked.');
   END IF;
   INSERT INTO Bookings (faculty_id, room_id, booking_date,
                         start_time, end_time, status_id)
   VALUES (p_user, p_room, p_date, p_start, p_end, 1);
   COMMIT;
END Smart_Book;
```

Cancel_Booking — ownership-checked cancellation

```
PROCEDURE Cancel_Booking(p_booking_id NUMBER,
                         p_user_id NUMBER, p_reason VARCHAR2) IS
   v_owner NUMBER;
BEGIN
   SELECT faculty_id INTO v_owner FROM Bookings
   WHERE booking_id = p_booking_id;
   IF v_owner != p_user_id THEN
      RAISE_APPLICATION_ERROR(-20003,'Unauthorized cancellation attempt.');
   END IF;
   UPDATE Bookings SET status_id = 3 WHERE booking_id = p_booking_id;
   INSERT INTO Cancellations (booking_id, cancelled_by, reason)
   VALUES (p_booking_id, p_user_id,
         NVL(p_reason,'Cancelled via PL/SQL Manager'));
   COMMIT;
END Cancel_Booking;
```

5.8 User-Defined Function

GET_FACULTY_WORKLOAD — Scalar function returning total teaching hours

```
CREATE OR REPLACE FUNCTION GET_FACULTY_WORKLOAD(p_faculty_id
NUMBER)
```

```
RETURN NUMBER
IS
   v_total_hours NUMBER := 0;
BEGIN
   SELECT NVL(SUM((end_time - start_time) * 24), 0)
   INTO v_total_hours
   FROM Bookings
   WHERE faculty_id = p_faculty_id AND status_id != 3;
   RETURN v_total_hours;
END;
```

In Oracle, subtracting two DATE values yields the difference in days. Multiplying by 24 converts this to hours. The function is used in a SELECT query to list all faculty with their computed workloads:

```
SELECT name, GET_FACULTY_WORKLOAD(2) AS total_hours_teaching
FROM Users WHERE role = 'FACULTY';
```

5.9 Representative SQL Queries

Q1: List all users with role names (JOIN)

```
SELECT u.user_id, u.name, u.email, r.role_name
FROM Users u
JOIN Roles r ON u.role_id = r.role_id
ORDER BY r.role_name, u.name;
```

Q2: All active bookings with faculty and room details (multi-table JOIN via view)

```
SELECT * FROM BookingDetails ORDER BY booking_date, start_time;
```

Q3: Students enrolled in DBS202 with their details

```
SELECT u.name, u.email
FROM Enrollments e
JOIN Users u ON e.student_id = u.user_id
WHERE e.course_id = 'DBS202';
```

Q4: Rooms with more than 2 bookings (GROUP BY + HAVING)

```
SELECT room_id, total_bookings
FROM RoomUsage
WHERE total_bookings > 2;
```

Q5: Conflict detection — overlapping bookings in same room (self-JOIN)

```
SELECT a.booking_id AS booking_A, b.booking_id AS booking_B,
      a.booking_date, a.start_time AS A_start, a.end_time AS A_end
FROM Bookings a
JOIN Bookings b ON a.room_id = b.room_id
   AND a.booking_id < b.booking_id
   AND a.booking_date = b.booking_date
   AND a.start_time < b.end_time
   AND a.end_time > b.start_time
WHERE a.status_id != 3 AND b.status_id != 3;
```

Q6: Courses with enrollments and booking activity (IN-subquery)

```
SELECT DISTINCT course_id FROM Bookings
WHERE status_id != 3
AND course_id IN (SELECT DISTINCT course_id FROM Enrollments);
```

Q7: Total bookings and cancellations per faculty (GROUP BY with aggregate)

```
SELECT u.name, COUNT(b.booking_id) AS total_bookings,
    SUM(CASE WHEN b.status_id = 3 THEN 1 ELSE 0 END) AS cancelled
FROM Users u
LEFT JOIN Bookings b ON u.user_id = b.faculty_id
WHERE u.role = 'FACULTY'
GROUP BY u.user_id, u.name
ORDER BY total_bookings DESC;
```

CHAPTER 6: RESULTS & SNAPSHOTS

This chapter presents the output of key queries and demonstrates the behaviour of triggers, stored procedures, views, and the web application dashboards. Placeholder boxes are provided for screenshots to be inserted.

6.1 Table Creation Output

	TABLE_NAME
1	BOOKINGS
2	BOOKINGSTATUSES
3	BOOKINGTYPES
4	BOOKING_AUDIT
5	CANCELLATIONS
6	CLASSROOMS
7	COURSES
8	DAYSOFWEEK
9	DEPARTMENTS
10	ENROLLMENTS
11	NOTIFICATIONS
12	NOTIFTYPES
13	PERIODICDETAILS
14	ROLES
15	SEMESTERS
16	USERS

6.2 View Outputs

BookingDetails view — active bookings with names

	BOOKING_ID	NAME	ROOM_NUMBER	BOOKING_DATE	START_TIME	END_TIME	STATUS_NAME
1	1	Dr. Arnab	101	08-04-26	08-04-26	08-04-26	Confirmed
2	2	Dr. Arnab	102	09-04-26	09-04-26	09-04-26	Confirmed
3	4	Prof. Sarah	301	09-04-26	09-04-26	09-04-26	Confirmed
4	6	Prof. Sarah	301	08-04-26	08-04-26	08-04-26	Confirmed

RoomUsage view — per-room booking count

	ROOM_ID	TOTAL_BOOKINGS
1	1	1
2	2	1
3	3	2

6.3 Trigger Verification

Trigger 1 — Room double-booking prevention

```
Error starting at line : 1 in command -
INSERT INTO Bookings(faculty_id,room_id,booking_date,start_time,end_time,status_id)
VALUES(1, 1, DATE '2025-06-01',
       TO_DATE('2025-06-01 09:30','YYYY-MM-DD HH24:MI'),
       TO_DATE('2025-06-01 10:30','YYYY-MM-DD HH24:MI'), 1)
Error at Command Line : 1 Column : 13
Error report -
SQL Error: ORA-20001: Room already booked for that time slot.
ORA-06512: at "SYSTEM.PREVENT_OVERLAP", line 13
ORA-04088: error during execution of trigger 'SYSTEM.PREVENT_OVERLAP'
```

Trigger 2 — Auto-notification on booking creation

	NOTIF_ID	USER_ID	MESSAGE	NOTIF_TYPE_ID	IS_READ	CREATED_AT
1	1	7	Booking Confirmed for course General in room 5	(null)	0	07-04-26 F40712:F420:31.905 PM

Trigger 3 — Faculty schedule conflict

```
Error starting at line : 3 in command -
INSERT INTO Bookings(faculty_id,room_id,booking_date,start_time,end_time,status_id)
VALUES(2, 1, DATE '2025-06-01',
       TO_DATE('2025-06-01 09:30','YYYY-MM-DD HH24:MI'),
       TO_DATE('2025-06-01 10:30','YYYY-MM-DD HH24:MI'), 1)
Error at Command Line : 3 Column : 13
Error report -
SQL Error: ORA-20004: Faculty schedule conflict: You are already teaching in another room during this exact time slot.
ORA-06512: at "SYSTEM.FACULTY_SCHEDULE_CONFLICT", line 13
ORA-04088: error during execution of trigger 'SYSTEM.FACULTY_SCHEDULE_CONFLICT'
```

Trigger 4 — Cancellation notification broadcast

	NOTIF_ID	USER_ID	MESSAGE	NOTIF_TYPE_ID	IS_READ	CREATED_AT
1	2	7	Your booking for room 5 has been cancelled.	(null)	0	07-04-26 F40712:F420:31.975 PM

Trigger 5 — Audit log on UPDATE

	AUDIT_ID	BOOKING_ID	ACTION	CHANGED_AT
1	1	1	UPDATE	07-04-26 F40712:F420:31.973 PM
2	2	1	UPDATE	07-04-26 F40712:F425:36.236 PM

6.4 Stored Procedure & Function Results

Generate_Course_Roster('DBS202') — Explicit cursor output

```
--- Course Roster for DBS202 ---
Student Name: Student Test | Email: student@manipal.edu
Student Name: Student 1 | Email: student1@manipal.edu
Student Name: Student 2 | Email: student2@manipal.edu
Student Name: Student 3 | Email: student3@manipal.edu
Student Name: Student 4 | Email: student4@manipal.edu
Student Name: Student 5 | Email: student5@manipal.edu
Student Name: Student 6 | Email: student6@manipal.edu
Student Name: Student 7 | Email: student7@manipal.edu
Student Name: Student 8 | Email: student8@manipal.edu
Student Name: Student 9 | Email: student9@manipal.edu
Student Name: Student 10 | Email: student10@manipal.edu
Student Name: Student 11 | Email: student11@manipal.edu
Student Name: Student 12 | Email: student12@manipal.edu
Student Name: Student 13 | Email: student13@manipal.edu
Student Name: Student 14 | Email: student14@manipal.edu
Student Name: Student 15 | Email: student15@manipal.edu
Student Name: Student 16 | Email: student16@manipal.edu
Student Name: Student 17 | Email: student17@manipal.edu
Student Name: Student 18 | Email: student18@manipal.edu
Student Name: Student 19 | Email: student19@manipal.edu
Student Name: Student 20 | Email: student20@manipal.edu
Student Name: Student 21 | Email: student21@manipal.edu
Student Name: Student 22 | Email: student22@manipal.edu
Student Name: Student 23 | Email: student23@manipal.edu
Student Name: Student 24 | Email: student24@manipal.edu
Student Name: Student 25 | Email: student25@manipal.edu
Student Name: Student 26 | Email: student26@manipal.edu
```

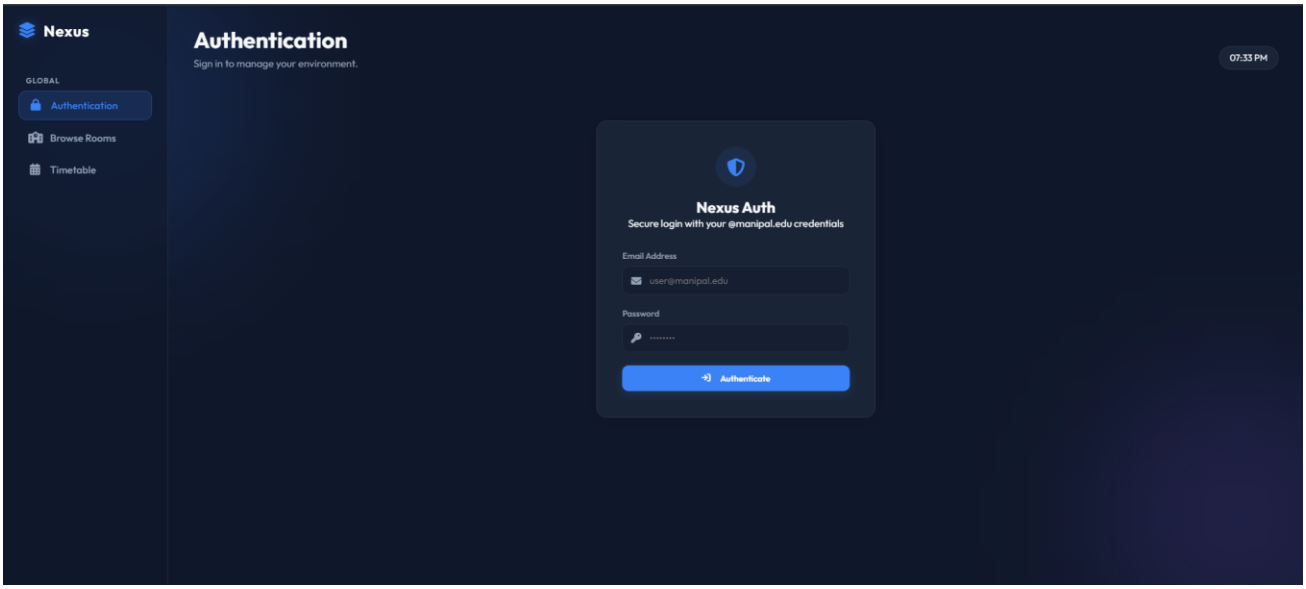
Get_Faculty_Workload — Scalar function query

	NAME	TOTAL_HOURS
1	Dr. Arnab	5
2	Prof. Sarah	4
3	Faculty 1	0
4	Faculty 2	0
5	Faculty 3	1
6	Faculty 4	0

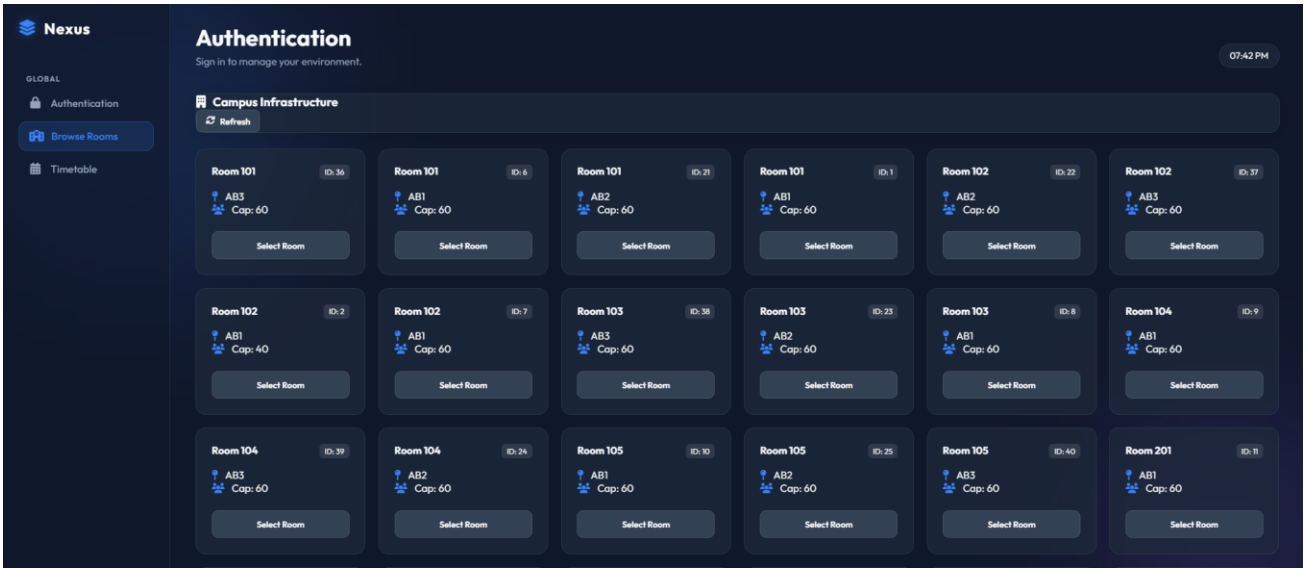
6.5 Web Application Snapshots

The web application is a single-page HTML/JS interface (Nexus) served at http://localhost:3000. It provides role-adaptive navigation that shows different portal sections depending on the logged-in user's role.

Login Page — Users authenticate with their @manipal.edu email and password. The server queries the Users table and returns the user_id, name, and role to the frontend. Role-specific navigation (Faculty Portal, Student Portal, Admin Gateway) is revealed after login.



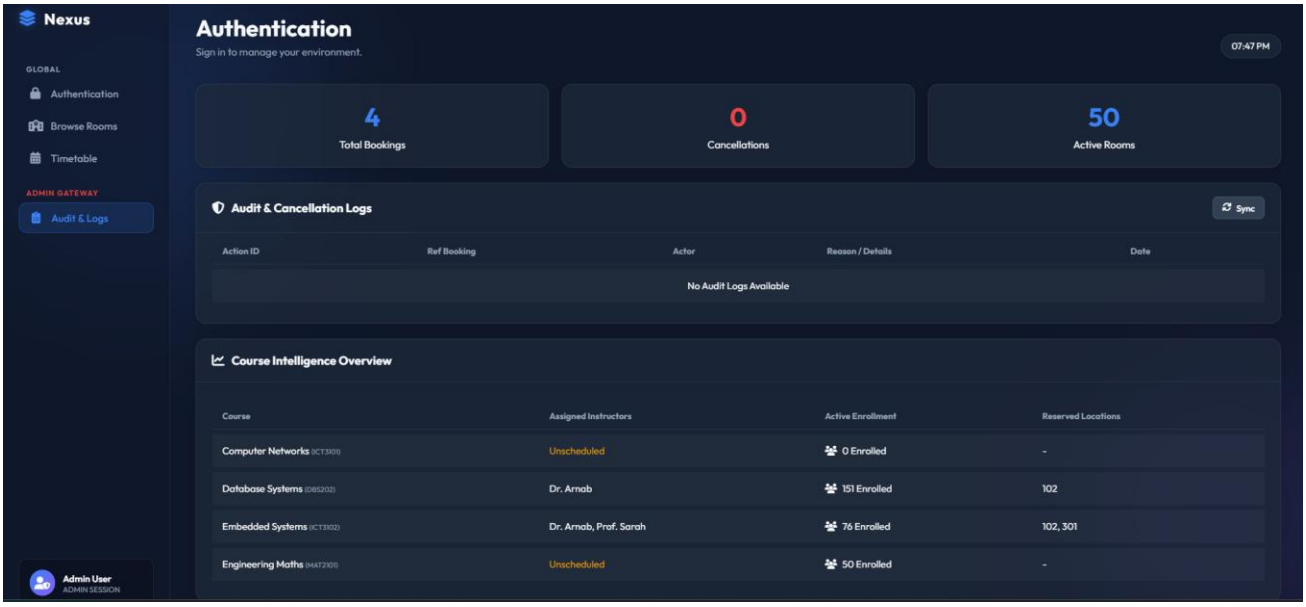
Browse Rooms — Shows all classrooms from the /rooms endpoint with building, room number, capacity, projector, and AC status as cards.



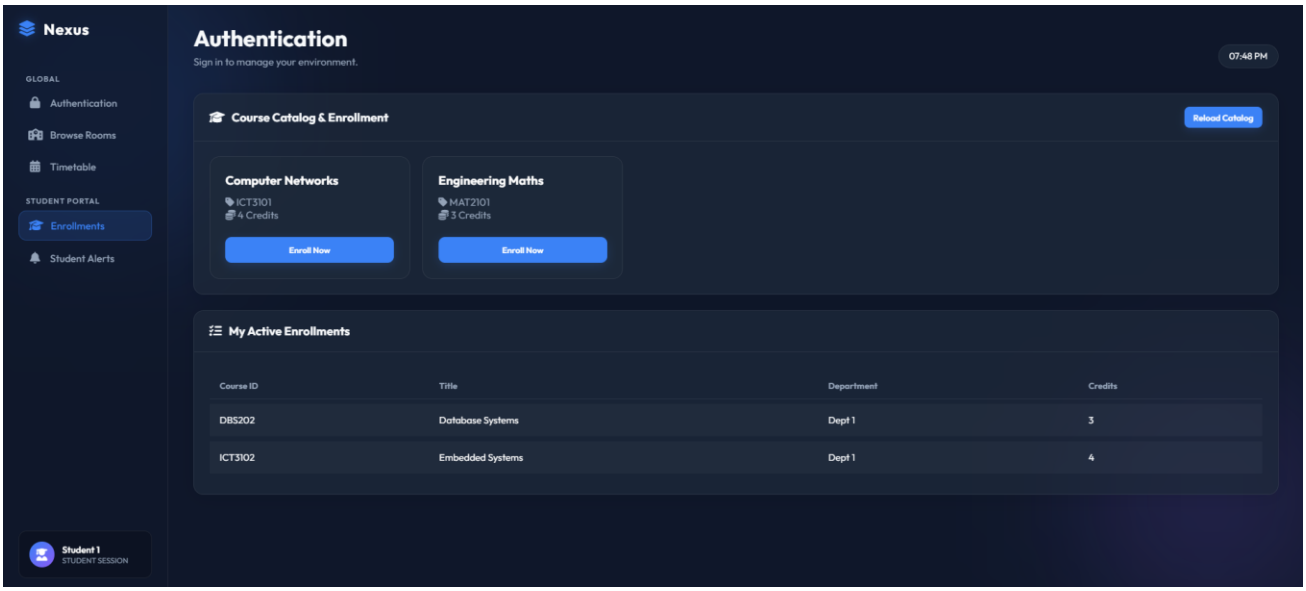
Timetable — Displays all active bookings from the /timetable endpoint. Recurring bookings are projected weekly in JavaScript — the server returns base booking rows, and the client loop generates additional projected occurrence rows up to the end_date.

Book Room (Faculty) — Faculty can create one-time or recurring bookings. Selecting 'Recurring Weekly' reveals the End Date field. The /book endpoint inserts into Bookings and conditionally inserts a PeriodicDetails row. Conflict errors from the prevent_overlap or faculty_schedule_conflict triggers surface as toast messages.

Admin Gateway — Shows total bookings, total cancellations, and total rooms from the /admin/stats endpoint. The Audit & Cancellation Logs table reads from the /admin/logs endpoint. The Course Intelligence Overview reads from /admin/courses-stats, showing enrolled student counts and faculty names per course (using Oracle LISTAGG).



Student Portal — Students can browse all courses and enroll via the /student/enroll endpoint. The Enrollments section shows active enrollments from the /student/courses endpoint. Student Alerts shows notifications from /notifications/:id.



CHAPTER 7: CONCLUSION, LIMITATIONS & FUTURE WORK

7.1 Conclusion

The Web-Based Classroom Booking System successfully demonstrates the design and full-stack implementation of a relational database application for managing academic room scheduling. The project achieved all stated objectives:

- A complete relational schema was designed and implemented in Oracle XE with 16 tables, all normalized to BCNF. The normalization process eliminated free-string domain columns, one transitive dependency, structural NULLs from periodic booking columns, and the repeating (semester, year) pattern.
- All 16 tables were created with appropriate PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK constraints. Oracle GENERATED BY DEFAULT AS IDENTITY syntax was used for auto-incrementing surrogate keys.
- Six triggers enforce business rules entirely at the database level — room conflict prevention, faculty schedule conflict prevention, automatic booking notifications, automatic cancellation notifications, periodic end-date validation, and booking update/delete auditing.
- The Booking_Manager PL/SQL package encapsulates Smart_Book and Cancel_Booking procedures. Generate_Course_Roster demonstrates explicit parameterized cursor handling. GET_FACULTY_WORKLOAD provides a reusable scalar computation.
- Three views (BookingDetails, RoomUsage, Dept_Capacity_Report) provide denormalized display data, with Dept_Capacity_Report demonstrating the ROLLUP grouping operator.
- A role-adaptive single-page web application demonstrates live Oracle connectivity via Node.js and the oracledb driver, with REST endpoints for all CRUD operations.
- Bulk data generation using PL/SQL loops creates 20 faculty and 150 students, with programmatic enrollment seeding ensuring realistic test data.

7.2 Limitations

- Password security: The database stores plain-text passwords. A production deployment must hash passwords using bcrypt before storage.
- Concurrent booking race condition: The conflict check inside Smart_Book and the prevent_overlap trigger are each correct in isolation, but under high concurrency two transactions could both pass the conflict check before either commits. Resolving this requires row-level locking (SELECT ... FOR UPDATE) on the target room-date combination.
- Periodic recurrence — no materialized occurrences: The system stores recurring bookings as a single Bookings row with a PeriodicDetails extension. Individual occurrence dates are projected in JavaScript on the client side, not stored in the database. This limits server-side per-date querying.
- Email/push notifications: Notifications are stored in the Notifications table and surfaced via the web UI. No actual email or push delivery mechanism is implemented.
- Admin-only booking: The application layer checks ownership (cancelled_by == faculty_id) for cancellations, but there is no database-level trigger enforcing that only faculty or admin roles can create bookings. Role enforcement relies on application logic only.
- Time zone handling: All DATE values are stored in the server's local Oracle time zone. Multi-campus deployments spanning time zones would require UTC storage.

7.3 Future Work

- Implement bcrypt password hashing in the Node.js layer and a password-change endpoint with verification.
- Add SELECT ... FOR UPDATE inside Smart_Book to eliminate the concurrent booking race condition at the database level.
- Implement a PL/SQL procedure to materialize recurring booking occurrences into a BookingOccurrences table, enabling per-date queries, attendance tracking, and individual-date rescheduling.
- Extend the admin dashboard with graphical room-utilization heat maps using the RoomUsage view data.
- Add a database-level BEFORE INSERT trigger on Bookings to enforce the faculty/admin role constraint directly in Oracle, independent of the application layer.
- Add WebSocket-based real-time notifications so students receive alerts immediately when a booking or cancellation trigger fires.
- Migrate to production deployment with environment-specific Oracle connection credentials via environment variables and connection pooling via oracledb.createPool().

CHAPTER 8: REFERENCES

- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.
- Ramakrishnan, R., & Gehrke, J. (2002). Database Management Systems (3rd ed.). McGraw-Hill.
- Oracle Database 21c / XE PL/SQL Language Reference. Oracle Corporation. https://docs.oracle.com/en/database/oracle/oracle-database/
- oracledb Node.js Driver Documentation. Oracle Corporation. https://node-oracledb.readthedocs.io/
- Express.js Documentation. https://expressjs.com/
- W3Schools SQL Tutorial. https://www.w3schools.com/sql/
- MDN Web Docs — Fetch API. Mozilla Foundation. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Addison-Wesley.